MATLAB-to-VLSI Design of a Fixed-Point FIR Filter

Shreyash Sharma

Contents

1 Introduction

Finite Impulse Response (FIR) filters are widely used in digital signal processing due to their inherent stability and linear phase characteristics. This project focuses on designing a 16-tap low-pass FIR filter in MATLAB and mapping it to a hardware-friendly fixed-point implementation suitable for RTL design.

The objective is to bridge algorithm-level design and hardware realization while analyzing trade-offs between performance, precision, and hardware cost.

2 FIR Filter Theory

The FIR filter output is given by:

$$y[n] = \sum_{k=0}^{N} b_k x[n-k] \tag{1}$$

where b_k are the filter coefficients and $x[n-k]$ are delayed input samples.

3 MATLAB Design

A 16-tap low-pass FIR filter was designed using the window method with a normalized cutoff frequency of 0.4.

3.1 Frequency Response

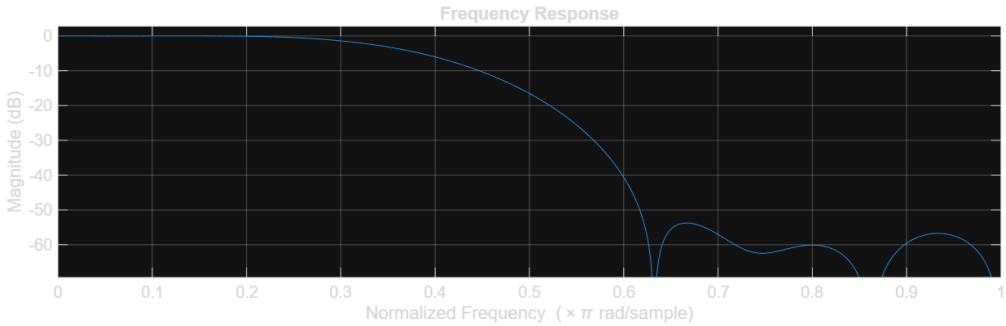

Figure 1: Frequency response of FIR filter

The frequency response shown in Fig. 1 demonstrates a flat passband, a transition region near the cutoff frequency, and strong attenuation in the stopband.

3.2 Time-Domain Filtering

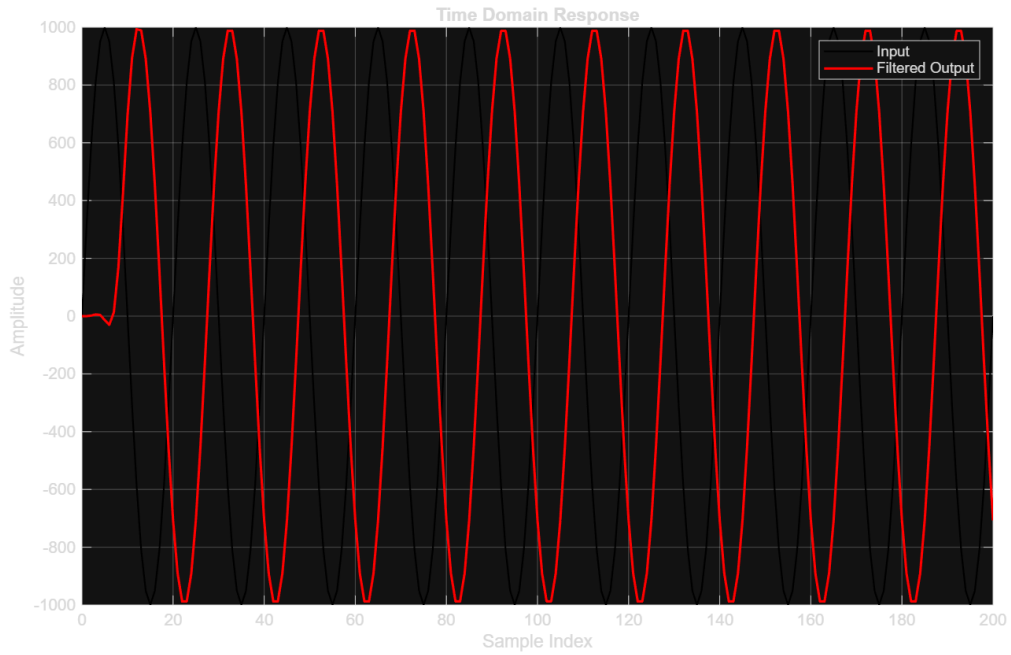

Figure 2: Noisy input vs filtered output

As shown in Fig. 2, the filter effectively removes high-frequency noise while preserving the desired signal.

3.3 Effect of Tap Count

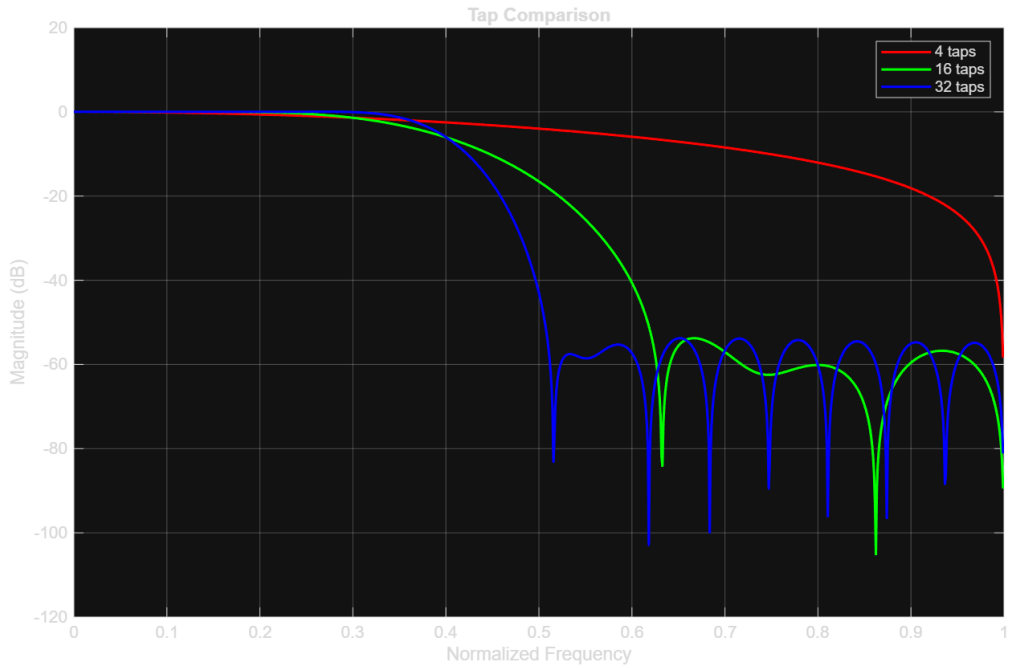

Figure 3: Comparison of 4, 16, and 32 tap filters

Fig. 3 shows that increasing the number of taps improves frequency selectivity but increases hardware complexity.

4 Fixed-Point Implementation

The floating-point coefficients were converted to Q1.15 fixed-point format by scaling with 2^{15} and rounding.

4.1 Coefficient Comparison

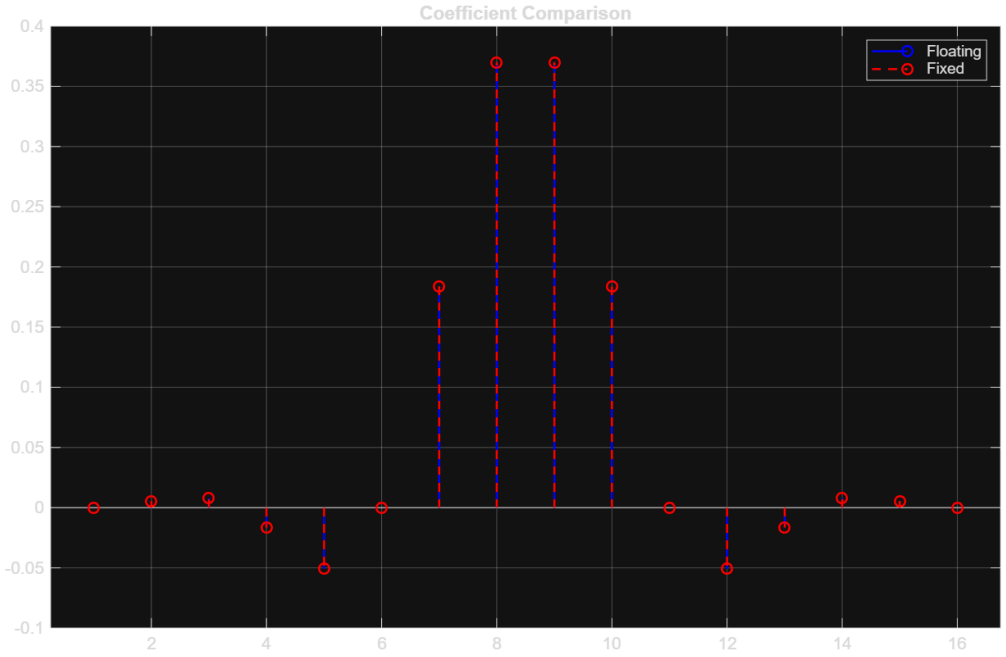

Figure 4: Floating-point vs fixed-point coefficients

4.2 Output Comparison

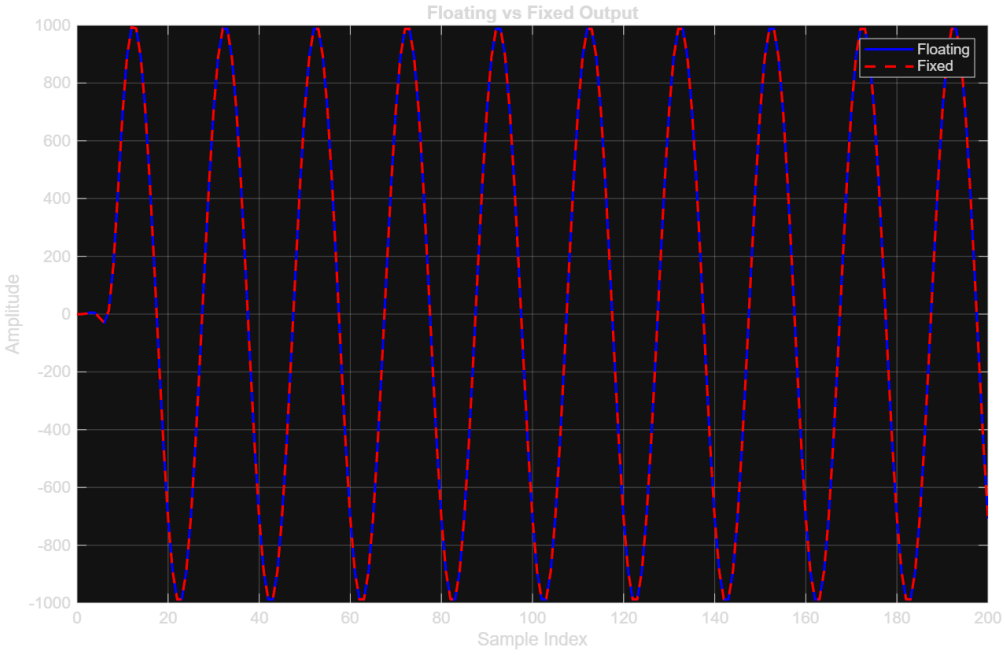

Figure 5: Floating vs fixed-point output

4.3 Quantization Error

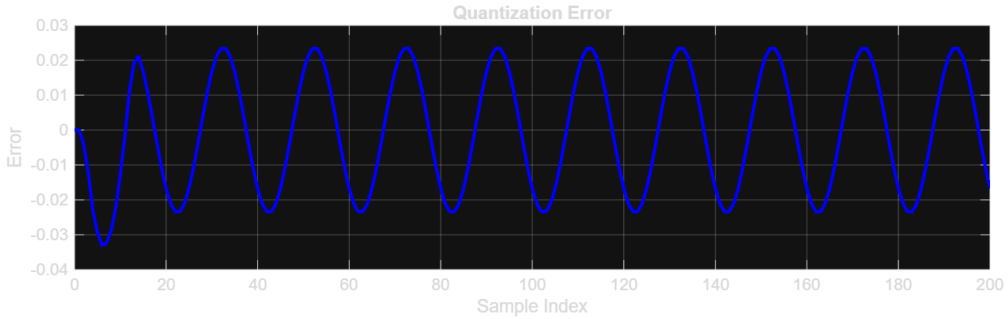

Figure 6: Quantization error

The quantization error shown in Fig. 6 is very small, indicating that fixed-point representation maintains high accuracy.

5 Hardware Architecture

5.1 FIR Datapath

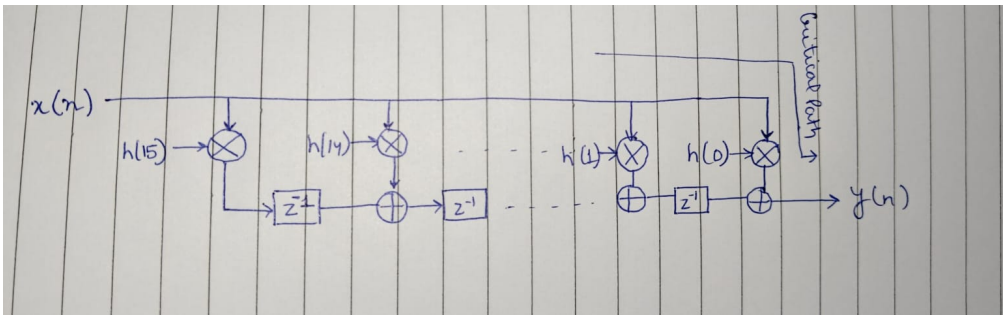

Figure 7: FIR filter hardware architecture

The architecture in Fig. 7 consists of shift registers, multipliers, and adders forming a multiply-accumulate structure.

5.2 RTL Architecture

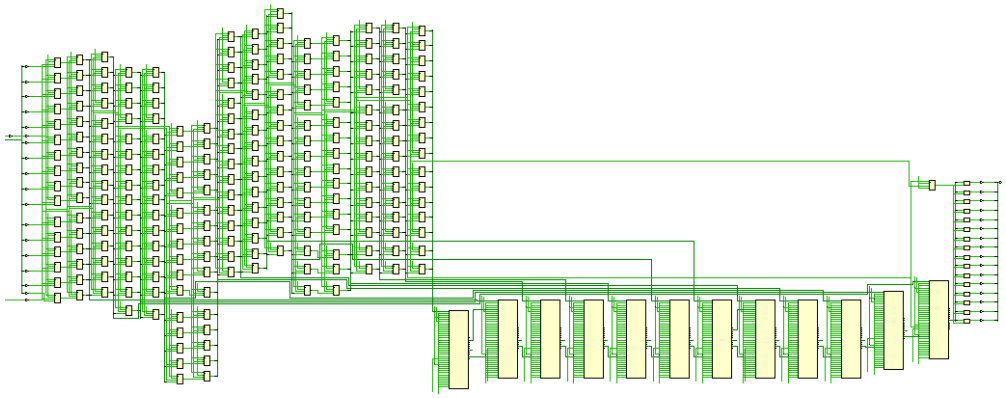

Figure 8: RTL datapath of FIR filter

As shown in Fig. 8, the RTL design uses clocked registers and combinational logic to produce one output per clock cycle.

6 Verilog Implementation

The FIR filter was implemented using a shift-register-based architecture in Verilog. Each clock cycle performs a multiply-accumulate operation, followed by scaling to maintain fixed-point precision.

7 Simulation Results

The RTL design was simulated in Vivado using a sinusoidal input with additive noise.

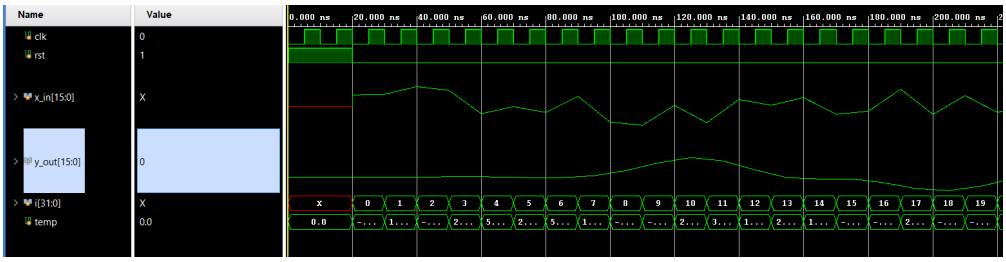

Figure 9: RTL simulation waveform

Fig. 9 shows the expected FIR filtering behavior with proper latency and smoothing.

8 Design Trade-offs

The design highlights key trade-offs:

- Increasing taps improves performance but increases hardware cost
- Fixed-point reduces hardware complexity but introduces quantization error
- Pipelining improves throughput at the cost of additional registers

9 Conclusion

A complete MATLAB-to-VLSI flow for FIR filter design was implemented. The filter was successfully designed, converted to fixed-point, and realized in RTL. Simulation results confirm correct functionality, demonstrating an effective balance between performance and hardware efficiency.